

# MUNICTECH EXPO 2026 — HANDS-ON TRIAL GUIDE

Run Your Own HAIEC Evaluation + A/B Release Comparison

## Purpose

This guide is for MunichTech Expo judges, reviewers, partners, and technical evaluators who want to try HAIEC themselves.

You can use it in two ways:

### JUDGE WORKSPACE NOTE

The dedicated MunichTech judge account is intentionally read-only for evaluation creation. Judges can inspect the frozen Kestrel system, Release A, Release B, comparison, reports, and evidence, but the server refuses new evaluation creation for the viewer role. The hands-on run instructions below are for a reviewer using their own authorized trial/workspace with evaluation access, not for the dedicated judge viewer account.

1. Quick product review — open the public KestrelVoice example and inspect the result.
2. Hands-on trial — connect your own GitHub repository and, when your trial/workspace includes Evaluations, run and compare two releases.

### IMPORTANT

The exact KestrelVoice source repository used for the MunichTech submission is private. The public report and consequence map can be viewed without source access, but an independent hands-on user should use a repository they control.

The product workflow you will reproduce is the same:

Connect source → Define AI system → Verify source → Run Evaluation → Inspect consequence paths → Run a second release → Compare evaluations.

## TECHNICAL ORIENTATION — WHAT THIS WORKFLOW IS PROVING

Not AI testing AI. HAIEC binds an evaluation to a source snapshot, qualifies the AI Action Path from capability to consequence, keeps Requested / Policy Authorized / Effectively Granted / Code Capable / Observed separate, compares qualified releases, and carries the evidence forward.

Use this mental model while following the steps:

MAP — Map the consequence.

PROVE — Prove the path.

ASSURE — Assure the boundary.

COMPARE — Compare the release.

CARRY — Carry the evidence.

Important evidence rule:

The conceptual unit of assurance is the AI Action Path. The AI Action Path Trail is the inspectable evidence representation of that path.

Approved visual references:

[https://drive.google.com/drive/folders/1xKC416UD1kru145Oz\\_GcW7iPnw\\_6Yxfg](https://drive.google.com/drive/folders/1xKC416UD1kru145Oz_GcW7iPnw_6Yxfg)

## MUNICTECH ACCESS AT A GLANCE

### IMPORTANT — WHAT A NEW SIGNUP RECEIVES TODAY

A normal new HAIEC signup does NOT automatically receive the highest product entitlement.

The current production signup path creates a 14-day trial with:

- tier = SCAN
- status = trialing

This applies to standard email signup and the normal OAuth signup paths for non-admin users.

So the normal public trial should be treated as a SCAN trial unless HAIEC has explicitly enabled broader MunichTech evaluation access for your account.

## PUBLIC REVIEW — NO ACCOUNT REQUIRED

Use the public KestrelVoice report and consequence map.

Best for:

Judges or reviewers who want to inspect the MunichTech submission immediately.

## STANDARD 14-DAY TRIAL — TRY HAIEC VALIDATION

Start here:

<https://www.haiec.com/signup>

You can use the normal 14-day trial to try HAIEC's standard validation experience, including the GitHub/source setup and the source and repository validation capabilities exposed to your trial account.

Typical path:

1. Create and verify your HAIEC account.
2. Connect GitHub.
3. Install the HAIEC GitHub App on a repository you control.
4. Define an AI System.
5. Connect and verify the repository.
6. Run the validation workflow available in your account.
7. Inspect the resulting source evidence and AI-system surfaces.

No local HAIEC software installation is required.

## FULL MUNICTECH RELEASE A/B COMPARISON

The full MunichTech comparison experience is:

Release A evaluation

→ Release B evaluation

→ Compare evaluations

→ inspect consequence/control change

→ inspect bounded / unresolved results

→ review Assurance

→ inspect portable evidence where available

First check your workspace.

If you can already open:

Evaluations

and

AI Systems → Reach & Impact → Compare evaluations

you may continue with the A/B steps in this guide. No separate access request is needed.

If those A/B comparison capabilities are not available in your trial, request complimentary MunichTech Expo evaluation access.

## HOW TO REQUEST COMPLIMENTARY MUNICTECH A/B ACCESS

Email:

subodhkc@subodhkc.com

Recommended subject:

MunichTech Expo — A/B Trial Access

Please include:

- the email address you used to sign up for HAIEC;
- your name or organization;
- the words “MunichTech Expo”;
- optionally, your GitHub username or the repository you plan to test.

If possible, send the request from the same email address used for your HAIEC signup.

Suggested message:

Hello,

I signed up for HAIEC with [your HAIEC signup email] after seeing the MunichTech Expo demo. I would like complimentary MunichTech evaluation access so I can try the Release A / Release B comparison workflow myself.

Name / organization: [optional]

GitHub username or repository: [optional]

Thank you.

HAIEC will confirm when the MunichTech evaluation access has been enabled.

This Expo access is intended to let reviewers try the standard product workflow shown in the MunichTech demo. It is not a certification, custom engineering engagement, or Enterprise POC.

## QUICK LINKS

Create account:

<https://www.haiec.com/signup>

Sign in:

<https://www.haiec.com/login>

Pricing / trial details:

<https://www.haiec.com/pricing>

Getting started:

<https://www.haiec.com/start-here>

Documentation:

<https://www.haiec.com/reference>

Acceptable Use Policy:

<https://www.haiec.com/legal/acceptable-use>

GitHub connection:

<https://www.haiec.com/dashboard/settings/github>

GitHub App dashboard:

<https://www.haiec.com/dashboard/github>

Install HAIEC GitHub App:

<https://github.com/apps/haiec-compliance/installations/new>

AI Systems:

<https://www.haiec.com/dashboard/ai-inventory>

Evaluation Runs:

<https://www.haiec.com/dashboard/audit-orchestrator>

Public KestrelVoice Assurance Report:

<https://www.haiec.com/sample-reports/kestrel>

Public KestrelVoice Consequence Map:

<https://www.haiec.com/sample-reports/kestrel/constellation>

Public Release A / Release B Comparison:

<https://www.haiec.com/sample-reports/kestrel/compare>

Official A/B-8 Judge Evidence Folder:

[https://drive.google.com/drive/folders/1S2gYsgRYr1PMq1fH\\_CDfxDjXOTbKO-dV](https://drive.google.com/drive/folders/1S2gYsgRYr1PMq1fH_CDfxDjXOTbKO-dV)

No local HAIEC software installation is required for the browser-based trial. The GitHub App is installed into your GitHub account or organization, not onto your computer.

## WHAT YOU NEED

Before starting, make sure you have:

- A HAIEC account.
- A GitHub account.
- Permission to install a GitHub App on the repository you want to evaluate.
- A repository you are authorized to scan.
- For the A/B comparison: two branches or release states in the same repository, such as:
  - main = baseline
  - candidate = changed release
- Desktop browser recommended.

Do not connect or scan a repository unless you are authorized to do so.

## PART A — QUICKLY VIEW THE MUNICTECH EXAMPLE

No GitHub setup is required for this section.

Public KestrelVoice Assurance Report:

<https://www.haiec.com/sample-reports/kestrel>

Public System Consequence Map:

<https://www.haiec.com/sample-reports/kestrel/constellation>

What to look for:

- Assurance disposition: REVIEW
- 44 consequential action paths in the evaluated path set
- 44 / 44 evaluated paths meeting the Code Capability criteria
- ORDER RECORD WRITE as a concrete consequence path
- Requested, Policy Authorized, Effectively Granted, Code Capable, and Observed kept as separate evidence questions
- Verified Action Frontier / evidence boundary

The public example is a frozen demonstration resu

## VISUAL REFERENCES — MUNICTECH EVIDENCE PACK

If you are reviewing the shared MunichTech Drive package, these screenshots show the expected KestrelVoice evidence surfaces before you begin your own trial.

Assurance decision:

<https://drive.google.com/file/d/1DQxEXPQ-ZjkfDIQnQCd07jCcghJdIpU8/view>

Assurance surface coverage:

<https://drive.google.com/file/d/1JNJpfofQTGKJ0ADtudOyiL6qG7xhwTb4/view>

System Consequence Map overview:

[https://drive.google.com/file/d/14TrrmQIUBtAFE381Q87ufqjLb4VOrw\\_I/view](https://drive.google.com/file/d/14TrrmQIUBtAFE381Q87ufqjLb4VOrw_I/view)

Focused ORDER RECORD WRITE path:

<https://drive.google.com/file/d/1-mv0G8YYwBLLPBPI2niY2pErZBPcqfXk/view>

Action proof evidence:

<https://drive.google.com/file/d/14WdbH4VKJDobIGFnXL4FWkJvMrpKi09B/view>

Executive report example:

<https://drive.google.com/file/d/1bjoRyU2drkyyYTShSYxCMh521xVdU4vc/view>

These are reference images from the frozen Kestrel Gold Evidence Pack. Your own repository will produce different paths, findings, and evidence states.

A/B-8 release-comparison evidence is now available in the Release Comparison Evidence folder:

[https://drive.google.com/drive/folders/1S2gYsgRYr1PMq1fH\\_CDfxDjXOTbKO-dV](https://drive.google.com/drive/folders/1S2gYsgRYr1PMq1fH_CDfxDjXOTbKO-dV)

It includes Release A / Release B views, comparison selection, CONTROL\_CHANGED, focused comparison evidence, UNRESOLVED ≠ ABSENT, evidence-plane views, portable evidence, the A/B-8 summary, manifest, and the packaged evidence ZIP.

To run your own evaluation, continue below.

## PART B — CONNECT GITHUB

### STEP 1 — SIGN IN TO HAIEC

1. If you do not have a HAIEC account, create one first:

<https://www.haiec.com/signup>

2. Complete any email-verification step shown during signup.

3. Sign in:

<https://www.haiec.com/login>

4. If you already use GitHub to sign in, you should still verify the GitHub connection from HAIEC Settings before scanning.

**Expected result:**

You arrive in the HAIEC dashboard.

### STEP 2 — CONNECT YOUR GITHUB ACCOUNT WITH OAUTH

**Navigation:**

Dashboard → Settings → GitHub

1. Open Settings.
2. Select GitHub.
3. On the GitHub Connection page, look for:  
GitHub Not Connected
4. Click:  
Connect GitHub

5. HAIEC redirects you to GitHub.
6. Authorize the requested GitHub connection.
7. Return to HAIEC.

**Expected result:**

The page should show:  
Connected to GitHub

It should also show the connected GitHub username.

**Why this is needed:**

GitHub OAuth links your GitHub identity to your HAIEC account. It is separate from installing the HAIEC GitHub App on a repository.

If it already says Connected to GitHub:  
You can continue to the next step.

## STEP 3 — INSTALL THE HAIEC GITHUB APP

**Navigation:**

Dashboard → GitHub App

The GitHub App is required for trusted repository verification in the AI System workflow.

1. Open:  
GitHub App
2. Click:  
Install on New Repo

If there are no installations yet, the page may instead show:  
Install GitHub App

3. GitHub opens the HAIEC Compliance App installation screen.
4. Choose the GitHub account or organization that owns the repository.
5. Select either:
  - All repositoriesOR

- Only select repositories

For an Expo test, selecting only the repository you plan to scan is simplest.

6. Finish the installation on GitHub.
7. Return to the HAIEC GitHub App page.
8. Click Refresh if the installation is not visible immediately.

**Expected result:**

You should see an active GitHub App installation and the selected repository count.

## STEP 4 — LINK THE GITHUB APP INSTALLATION TO YOUR HAIEC ORGANIZATION

If the GitHub App card says:  
Organization linkage required

then:

1. Open the organization dropdown.
2. Select the HAIEC organization/workspace you are using.
3. Click:  
Link Organization

**Expected result:**

The installation should show:  
Linked to organization

If it is already linked, skip this step.

### WHY BOTH OAUTH AND THE GITHUB APP?

They serve different purposes:

**GitHub OAuth**

→ links your GitHub identity to HAIEC.

**GitHub App installation**

→ gives HAIEC repository-scoped trusted access to the repositories you select.

For the source-verification path used by the MunichTech demo, the repository must be observable through an organization-linked GitHub App installation.

## PART C — CREATE YOUR AI SYSTEM

### STEP 5 — OPEN AI SYSTEMS

**Navigation:**

Dashboard → AI Systems

The page title should be:

AI Systems

If this is your first system, you may see:

Start your first HAIEC evaluation

Click:

Add system to HAIEC

If you already have systems, click:

Define AI System

### STEP 6 — DEFINE THE SYSTEM

HAIEC opens:

Step 1: Define Your AI System

Complete the fields:

System name

Required.

Example:

MunichTech Demo Agent

What are you evaluating?

Required.

For most agentic application demos, choose one of:

- AI Application
- AI Agent

Choose the option that best describes your project.

Provider

Optional.

Examples:

OpenAI

Anthropic

Google

Custom

Environment

Required.

Choose the environment that accurately describes the repository/system.

Description

Optional.

Example:

Agentic application used to demonstrate source-backed action assurance and release comparison.

Click:

Create System

**Expected result:**

HAIEC creates the AI System and advances to:

Step 2: Connect Where It Lives

## PART D — CONNECT THE REPOSITORY TO THE AI SYSTEM

## STEP 7 — ADD YOUR REPOSITORY AS THE SYSTEM SOURCE

On:

Step 2: Connect Where It Lives

Find:

Where can HAIEC observe or evaluate it?

Choose:

Code Repository

Description: GitHub/GitLab repository

Enter:

Asset name

Example:

my-org/my-agent-repo

Locator (URL/ID)

Use the full GitHub repository URL.

Example:

<https://github.com/my-org/my-agent-repo>

Click:

Connect Asset

### **Important:**

At this point, HAIEC has REGISTERED the source.

REGISTERED does not yet mean CONNECTED or VERIFIED.

Click:

Open System Workspace

## STEP 8 — VERIFY THE SOURCE

Inside the AI System workspace:

### **Navigation:**

AI Systems → your system → Sources

Find the repository you just added.

If it is not already:

CONNECTED + VERIFIED

look for:

Verify source

Click:

Verify source

HAIEC checks the repository against the GitHub App installation linked to your HAIEC organization.

**Expected result:**

The source should show both:

Connection:

Connected

Identity:

Verified

If the button says:

Admin verification required

an HAIEC organization owner/admin must perform the verification.

If verification fails:

Check that:

- the repository URL is correct;
- the GitHub App is installed on that repository;
- the GitHub App installation is linked to the same HAIEC organization;
- your GitHub connection is active.

Do not continue to the evaluation until the intended source is verified.

## ACCESS CHECK BEFORE CONTINUING

The steps from this point use the Evaluation / Assurance workflow.

Continue if your HAIEC workspace visibly allows you to open Evaluations and start a new Evaluation for the AI System.

If the workflow is locked in your account, stop here rather than bypassing the lock. You can still complete the standard validation steps available in your trial. To try the full MunichTech Release A / Release B comparison, email [subodhkc@subodhkc.com](mailto:subodhkc@subodhkc.com) with your HAIEC signup email and mention “MunichTech Expo” to request complimentary evaluation access. Wait for confirmation before continuing with the A/B steps.

## PART E — RUN YOUR FIRST EVALUATION

### STEP 9 — PREPARE YOUR BASELINE BRANCH

For a normal single evaluation, use the branch you want to review.

For an A/B test, prepare two distinct release states.

Simple example:

**Baseline:**

main

**Candidate:**

candidate

The branches should represent two real source states of the same AI system.

For a meaningful consequence comparison, change something consequential in the candidate, for example:

- add or remove an AI-facing action;
- change a control or confirmation requirement;
- change a downstream resource/action path.

Also consider including a non-consequential change so you can observe whether HAIEC avoids promoting noise into a consequence claim.

Do not use two branches from different unrelated systems.

## STEP 10 — START RELEASE A / BASELINE EVALUATION

From the system workspace, click:  
Run Evaluation

Alternative navigation:  
Dashboard → Evaluation Runs → New Evaluation

The page should show:  
Evaluate [your system name]

Confirm:

System  
Your intended AI System is selected.

Source  
Your verified repository is selected.

Branch to evaluate  
Enter your baseline branch.

Example:  
main

If only one eligible verified repository exists, HAIEC may preselect it.

For private repositories, HAIEC may require repository consent before the Run Evaluation button becomes available.

If prompted:  
grant repository access for the selected branch.

For the standard MunichTech-style source-backed run:  
leave Advanced Evidence closed unless you intentionally want to attach other evidence.

Click:  
Run Evaluation

## STEP 11 — WATCH THE RUN

### **Navigation:**

Dashboard → Evaluation Runs

You will see the run in the run list.

A healthy static evaluation proceeds through the canonical lifecycle:

PENDING

→ SCANNING

→ ANALYZING

→ COMPLETED

You do not need to keep the page open continuously.

Once the run is complete, it should appear under:

Completed

### **Important:**

A completed Evaluation Run is operational evidence input. The canonical Assurance result is viewed in the AI System / Assurance surfaces.

## PART F — REVIEW THE FIRST EVALUATION

## STEP 12 — RETURN TO THE AI SYSTEM

### **Navigation:**

AI Systems → your system

Useful tabs include:

Overview

Sources

Evidence

Assurance

Reports & Receipts

Advanced:

Reach & Impact  
Action & Authority  
Repository Intelligence  
Architecture  
Activity

## STEP 13 — OPEN REACH & IMPACT

Navigate:  
AI Systems → your system → Reach & Impact

This is the main MunichTech technical demo surface.

At the top, HAIEC may show:

Current source  
Latest Evaluation

Use:  
Latest Evaluation

when you want the map tied to the exact evidence snapshot of the completed evaluation.

This prevents current repository state from being silently mixed with historical evaluated evidence.

## STEP 14 — INSPECT THE SYSTEM CONSEQUENCE MAP

The map answers:  
What actions and downstream consequences can this AI system reach?

Select a consequential path.

Look for the chain from:

AI-facing capability  
→ dispatch / implementation  
→ handler  
→ resource  
→ consequence

Depending on your application, examples may include:

- database write
- record creation
- external API action
- configuration change
- message/send operation
- persistent state change

The exact KestrelVoice example reaches:  
ORDER RECORD WRITE

Your repository will produce its own evidence and consequences.

Important interpretation:

Code Capable  
does not mean  
Policy Authorized

and it does not mean:  
Observed execution.

## PART G — RUN RELEASE B / CANDIDATE

### STEP 15 — START THE SECOND EVALUATION

Go back to the same AI System.

Click:  
Run Evaluation

Use the SAME:

- HAIEC organization
- AI System
- repository source

Change only the branch/release to the candidate state.

Example:

**Baseline:**

main

**Candidate:**

candidate

On:

Branch to evaluate

enter your candidate branch.

Click:

Run Evaluation

Wait for it to reach:

COMPLETED

You now have two completed evaluations of the same AI system.

## PART H — COMPARE RELEASE A AND RELEASE B

### STEP 16 — OPEN THE COMPARISON UI

Navigate:

AI Systems

→ your system

→ Reach & Impact

Click:

Compare evaluations

The panel opens:

What changed between Evaluations?

You should see two selectors:

Baseline (reference)

Target

## STEP 17 — SELECT THE TWO COMPLETED EVALUATIONS

For Baseline:

select the Release A evaluation.

For Target:

select the Release B evaluation.

HAIEC labels evaluations using the date, disposition, and the beginning of the evaluation ID.

Double-check the order before comparing.

Baseline

should be the earlier/reference release.

Target

should be the candidate/new release.

Click:

Compare

## STEP 18 — READ CONSEQUENCE DELTA

HAIEC can classify supported release changes such as:

New consequence

No longer established

Expanded reach

Narrowed reach

Control changed

Dependency changed

Evidence changed

Unresolved

The panel is evidence-bound.

**Important:**

HAIEC does not treat every code diff as a consequence change.

The central MunichTech idea is:

Git diff shows what code changed.

HAIEC shows what changed in consequence.

## STEP 19 — USE THE MAP OVERLAY

When a comparison item can be mapped to the candidate consequence graph:

1. Click the comparison item.
2. HAIEC focuses the corresponding candidate map node/path.
3. Inspect where the changed consequence/control appears in the system path.

This lets you move from:

release-level change

to

the affected consequence path.

## STEP 20 — UNDERSTAND UNRESOLVED RESULTS

If HAIEC shows:

Unresolved

that does not automatically mean the system is safe, unchanged, added, or removed.

One important rule is:

**UNRESOLVED ≠ ABSENT**

Another is:

**PARTIAL COVERAGE ≠ PROOF OF ABSENCE**

This is intentional.

If the evidence is not strong enough to establish that something disappeared or was newly added, HAIEC should leave the claim unresolved rather than manufacture certainty.

## PART I — REVIEW ASSURANCE AND PORTABLE EVIDENCE

### STEP 21 — OPEN ASSURANCE

**Navigation:**

AI Systems → your system → Assurance

Review the evidence state and current disposition.

HAIEC keeps these evidence questions separate:

Requested  
Policy Authorized  
Effectively Granted  
Code Capable  
Observed

Do not interpret one plane as proof of another.

### STEP 22 — OPEN REPORTS & RECEIPTS

**Navigation:**

AI Systems → your system → Reports & Receipts

Depending on the available evidence and account state, you can inspect or issue artifacts such as:

- Assurance Report
- Decision Receipt
- portable evidence/package outputs

If the UI offers:

Issue Decision Receipt

you can issue a receipt for the canonical completed evaluation.

A Decision Receipt is a release evidence artifact.  
It is not a certification.

## PART J — MUNICTECH RECOMMENDED 5-MINUTE SELF-GUIDED PATH

If you already have two completed evaluations and want the fastest hands-on review:

1. Sign in.
2. Open AI Systems.
3. Open your test system.
4. Open Reach & Impact.
5. Switch to Latest Evaluation.
6. Inspect one source-backed consequence path.
7. Click Compare evaluations.
8. Select Release A as Baseline.
9. Select Release B as Target.
10. Click Compare.
11. Open one established change.
12. Open one unresolved change if available.
13. Open Assurance.
14. Open Reports & Receipts.

The intended story is:

### **MAP**

→ What can the system reach?

### **PROVE**

→ What source-backed path reaches the consequence?

### **ASSURE**

→ **What evidence is established, and what strengthens assurance next?**

### **COMPARE**

→ What changed between two evaluated releases?

### **CARRY**

→ What evidence travels with the release?

## PART K — IF YOU WANT TO RECREATE THE MUNICTECH A/B PATTERN

The exact KestrelVoice source repository used for the submission is private, so public users cannot reproduce the exact frozen A/B pair without explicit repository access.

You can recreate the same experimental shape on your own repository.

Create:

Release A

A baseline branch.

Release B

A candidate branch derived from A.

Useful controlled changes for Release B:

1. Add a consequential capability.

Example:

Expose a new agent tool that reaches a durable write.

2. Remove or narrow an existing capability.

Example:

Remove the action from model/tool exposure while leaving dormant implementation code.

3. Change a control.

Example:

Require confirmation before dispatching a consequential action.

4. Add a non-consequential change.

Example:

Documentation or experiment metadata only.

Then:

Run A

→ complete evaluation

Run B

→ complete evaluation

Reach & Impact

→ Compare evaluations

Evaluate whether HAIEC:

- establishes supported consequence/control changes;
- leaves unsupported absence claims unresolved;
- avoids promoting non-consequential noise into established structural changes.

## PART L — TROUBLESHOOTING

“GitHub Not Connected”

Go to:

Settings → GitHub → Connect GitHub

“Repository does not verify”

Check:

- GitHub App installed?
- Correct repository selected in GitHub App installation?
- Installation linked to the correct HAIEC organization?
- Repository URL correct?

Then retry:

Sources → Verify source

“Admin verification required”

An organization owner/admin must verify the repository source.

“Run Evaluation is disabled”

Check:

- correct AI System selected;
- static source evidence enabled;
- verified source selected;
- branch entered;
- repository consent ready.

“No evaluations appear under Compare evaluations”

The comparison UI only lists completed Assurance evaluations for that AI System.

You need at least two completed evaluations.

“Comparison not available”

The two evaluations may not be safely comparable. Read the limitation message shown by HAIEC.

“Why is a change unresolved?”

The evidence may not be sufficient to prove addition/removal or exact cross-release identity. HAIEC intentionally fails closed rather than guessing.

“Current source looks different from the evaluation”

Use:

Reach & Impact → Latest Evaluation

Current source and historical evaluated source are intentionally separate.

## PART M — WHAT THE MUNICTECH DEMO PROVES

The MunichTech submission is not simply demonstrating a scanner.

It demonstrates a release assurance workflow:

Exact source snapshot

→ source-backed action path

→ consequential effect

→ separate authority/evidence planes

→ release-to-release consequence comparison

→ portable release evidence

The central question is:

WHAT CAN YOUR AI ACTUALLY CAUSE?

HAIEC

Human AI Evidence Company

High assurance in every consequence.